

```
// === ДОБАВЛЯЕМ ОБЯЗАТЕЛЬНУЮ ВЕРШИННУЮ ТОЧКУ ВХОДА ДЛЯ DIRECTX 11 ===
```

```
struct VS_OUTPUT
```

```
{
```

```
    float4 Pos : SV_POSITION;
```

```
    float2 Tex : TEXCOORD0;
```

```
};
```

```
VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
```

```
{
```

```
    VS_OUTPUT output = (VS_OUTPUT)0;
```

```
    output.Pos = Pos;
```

```
    output.Tex = Tex;
```

```
    return output;
```

```
}
```

```
// =====
```

```
// ИСТИННЫЙ ОДНОПРОХОДНЫЙ ДЕФЕКТОСКОП АНАТОЛИЯ ФЕДОРОВИЧА (ОБРАЗЕЦ 2011 ГОДА)
```

```
// =====
```

```
// Входные ресурсы DirectX 11
```

```
Texture2D srcTexture : register(t0);    // Наш единственный сырой файл
```

```
SamplerState samplerState0 : register(s0); // Point-сэмплер без замыливания
```

```
// Буфер констант (бывший register(c0))
```

```
cbuffer cbData : register(b0)
```

```
{
```

```
    float width;    // p0[0]
```

```
    float height;   // p0[1]
```

```
    float d_width;  // p0[2]
```

```
    float d_height; // p0[3]
```

```
    float splitX;   // Положение шторки
```

```
    float padding;
```

```
};
```

```
static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);
```

```
// Вспомогательная функция чтения и перевода пикселя в монохром (tex2D)
```

```
float GetGray(float2 uv)
```

```
{
```

```
    return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
```

```
}
```

```
// 1. ШЕЙДЕР ДЛЯ ЛЕВОГО ВЕРХНЕГО УЧАСТКА
```

```
float4 func_LT(float2 tex_in)
```

```
{
```

```
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
```

```
    float2 tex_8 = (tex_in - 4.0f * d_xy) / 8.0f;
```

```
    float2 tex_in_3 = d_xy + tex_8 + float2(0.25f, 0.75f);
```

```
// === ИСПРАВЛЕНИЕ ФАНТОМОВ В SHADERS.HLSL (ДЛЯ ФУНКЦИЙ ЗОН) ===
```

```
// 1. Принудительно привязываем текущую координату к жесткой сетке макроблоков 4x4
```

```
float2 block_uv = floor(tex_in * float2(width, height) / 4.0f) * 4.0f / float2(width, height);
```

```
// 2. Считываем соседей строго от выровненной точки block_uv, добавляя шаг по центру соседних блоков
```

```
float2 tex_LR = block_uv + float2(d_width * 6.0f, d_height * 2.0f);
```

```
float2 tex_TB = block_uv + float2(d_width * 2.0f, d_height * 6.0f);
```

```
float2 tex_LRTB = block_uv + float2(d_width * 6.0f, d_height * 6.0f);
```

```
float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
```

```
float k_S = GetGray(tex_in_3);
```

```
float k_LR = GetGray(tex_LR);
```

```
float k_TB = GetGray(tex_TB);
```

```
float k_LRTB = GetGray(tex_LRTB);
```

```

int i_x = round((tex_in + 4.0f * d_xy) * width);
int i_y = round((tex_in + 4.0f * d_xy) * height);
i_x = (i_x % 4) * 2;
i_y = (i_y % 4) * 2;

// Коэффициенты: 9 и 7
float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
              (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

/*
// === НАЧАЛО ИЗМЕНЕНИЯ: ПОЛНОЦЕННЫЙ ДИСПЕРСИОННЫЙ ДЕМПФЕР ИЗ 2011 ГОДА ===

float macro_mid = k_SS;    // Интерполированный фон 4x4
float global_mid = k_S;    // В качестве общего фона 8x8 используем центральный блок

// 1. Вычисляем разницу исходного пикселя и фона
float delta = k1 - macro_mid; // k1 — наш исходный монохромный пиксель

// 2. Считаем локальную дисперсию (модуль разности двух фонов)
float local_dispersion = abs(macro_mid - global_mid);

// 3. Защитный знаменатель из паспорта (12.75 / 255.0f = 0.05f в диапазоне DirectX)
float denom = local_dispersion + 0.05f;

// 4. Локальный контраст (деление на дисперсию, которое убьет фантомы)
float local_contrast = delta / denom;

// 5. Финальный синтез яркости с подгоночным коэффициентом CPU (1.8f) и сдвигом
float out_gray = saturate(macro_mid + local_contrast * (local_dispersion * 1.8f));
// === КОНЕЦ ИЗМЕНЕНИЯ ===
*/
k_SS = round(k_SS * 255.0f) / 255.0f;

```

```

float out_gray = saturate(k1 - k_SS + 0.35f);

return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 2. ШЕЙДЕР ДЛЯ ПРАВОГО ВЕРХНОГО УЧАСТКА
float4 func_RT(float2 tex_in)
{
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
    float2 tex_8 = (tex_in - 4.0f * d_xy) / 8.0f;
    float2 tex_in_3 = d_xy + tex_8 + float2(0.25f, 0.75f);

    float2 tex_LR = tex_in_3 + float2(d_width, 0.0f);
    float2 tex_TB = tex_in_3 - float2(0.0f, d_height);
    float2 tex_LRTB = tex_in_3 + float2(d_width, -d_height);

    float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

    int i_x = round((tex_in + 4.0f * d_xy) * width);
    int i_y = round((tex_in + 4.0f * d_xy) * height);
    i_x = (i_x % 4) * 2;
    i_y = (i_y % 4) * 2;

    // Коэффициенты: 15, 1, 9, 7
    float k_SS = ((k_S * (15 - i_x) + k_LR * (1 + i_x)) * (9 + i_y) +
        (k_TB * (15 - i_x) + k_LRTB * (1 + i_x)) * (7 - i_y)) / 256.0f;

    /*

```

```

// === НАЧАЛО ИЗМЕНЕНИЯ: ПОЛНОЦЕННЫЙ ДИСПЕРСИОННЫЙ ДЕМПФЕР ИЗ 2011 ГОДА ===

float macro_mid = k_SS;    // Интерполированный фон 4x4
float global_mid = k_S;    // В качестве общего фона 8x8 используем центральный блок

// 1. Вычисляем разницу исходного пикселя и фона
float delta = k1 - macro_mid; // k1 — наш исходный монохромный пиксель

// 2. Считаем локальную дисперсию (модуль разности двух фонов)
float local_dispersion = abs(macro_mid - global_mid);

// 3. Защитный знаменатель из паспорта (12.75 / 255.0f = 0.05f в диапазоне DirectX)
float denom = local_dispersion + 0.05f;

// 4. Локальный контраст (деление на дисперсию, которое убьет фантомы)
float local_contrast = delta / denom;

// 5. Финальный синтез яркости с подгоночным коэффициентом CPU (1.8f) и сдвигом
float out_gray = saturate(macro_mid + local_contrast * (local_dispersion * 1.8f));

// === КОНЕЦ ИЗМЕНЕНИЯ ===

*/
k_SS = round(k_SS * 255.0f) / 255.0f;
float out_gray = saturate(k1 - k_SS + 0.35f);

return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 3. ШЕЙДЕР ДЛЯ ПРАВОГО НИЖНЕГО УЧАСТКА
float4 func_RB(float2 tex_in)
{
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
    float2 tex_8 = (tex_in - 4.0f * d_xy) / 8.0f;
    float2 tex_in_3 = d_xy + tex_8 + float2(0.25f, 0.75f);

```

```

float2 tex_LR = tex_in_3 + float2(d_width, 0.0f);
float2 tex_TB = tex_in_3 + float2(0.0f, d_height);
float2 tex_LRTB = tex_in_3 + float2(d_width, d_height);

float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
float k_S = GetGray(tex_in_3);
float k_LR = GetGray(tex_LR);
float k_TB = GetGray(tex_TB);
float k_LRTB = GetGray(tex_LRTB);

int i_x = round((tex_in + 4.0f * d_xy) * width);
int i_y = round((tex_in + 4.0f * d_xy) * height);
i_x = (i_x % 4) * 2;
i_y = (i_y % 4) * 2;

// Коэффициенты: 15, 1, 15, 1
float k_SS = ((k_S * (15 - i_x) + k_LR * (1 + i_x)) * (15 - i_y) +
              (k_TB * (15 - i_x) + k_LRTB * (1 + i_x)) * (1 + i_y)) / 256.0f;

/*
// === НАЧАЛО ИЗМЕНЕНИЯ: ПОЛНОЦЕННЫЙ ДИСПЕРСИОННЫЙ ДЕМПФЕР ИЗ 2011 ГОДА ===
float macro_mid = k_SS; // Интерполированный фон 4x4
float global_mid = k_S; // В качестве общего фона 8x8 используем центральный блок

// 1. Вычисляем разницу исходного пикселя и фона
float delta = k1 - macro_mid; // k1 — наш исходный монохромный пиксель

// 2. Считаем локальную дисперсию (модуль разности двух фонов)
float local_dispersion = abs(macro_mid - global_mid);

// 3. Защитный знаменатель из паспорта (12.75 / 255.0f = 0.05f в диапазоне DirectX)

```

```

float denom = local_dispersion + 0.05f;

// 4. Локальный контраст (деление на дисперсию, которое убьет фантомы)
float local_contrast = delta / denom;

// 5. Финальный синтез яркости с подгоночным коэффициентом CPU (1.8f) и сдвигом
float out_gray = saturate(macro_mid + local_contrast * (local_dispersion * 1.8f));

// === КОНЕЦ ИЗМЕНЕНИЯ ===

*/

k_SS = round(k_SS * 255.0f) / 255.0f;
float out_gray = saturate(k1 - k_SS + 0.35f);

return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 4. ШЕЙДЕР ДЛЯ ЛЕВContextОГО НИЖНЕГО УЧАСТКА
float4 func_LB(float2 tex_in)
{
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
    float2 tex_8 = (tex_in - 4.0f * d_xy) / 8.0f;
    float2 tex_in_3 = d_xy + tex_8 + float2(0.25f, 0.75f);

    float2 tex_LR = tex_in_3 - float2(d_width, 0.0f);
    float2 tex_TB = tex_in_3 + float2(0.0f, d_height);
    float2 tex_LRTB = tex_in_3 - float2(d_width, -d_height);

    float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

```

```

int i_x = round((tex_in + 4.0f * d_xy) * width);
int i_y = round((tex_in + 4.0f * d_xy) * height);
i_x = (i_x % 4) * 2;
i_y = (i_y % 4) * 2;

// Коэффициенты: 9, 7, 15, 1
float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (15 - i_y) +
              (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (1 + i_y)) / 256.0f;

/*
// === НАЧАЛО ИЗМЕНЕНИЯ: ПОЛНОЦЕННЫЙ ДИСПЕРСИОННЫЙ ДЕМПФЕР ИЗ 2011 ГОДА ===

float macro_mid = k_SS;    // Интерполированный фон 4x4
float global_mid = k_S;    // В качестве общего фона 8x8 используем центральный блок

// 1. Вычисляем разницу исходного пикселя и фона
float delta = k1 - macro_mid; // k1 — наш исходный монохромный пиксель

// 2. Считаем локальную дисперсию (модуль разности двух фонов)
float local_dispersion = abs(macro_mid - global_mid);

// 3. Защитный знаменатель из паспорта (12.75 / 255.0f = 0.05f в диапазоне DirectX)
float denom = local_dispersion + 0.05f;

// 4. Локальный контраст (деление на дисперсию, которое убьет фантомы)
float local_contrast = delta / denom;

// 5. Финальный синтез яркости с подгоночным коэффициентом CPU (1.8f) и сдвигом
float out_gray = saturate(macro_mid + local_contrast * (local_dispersion * 1.8f));

// === КОНЕЦ ИЗМЕНЕНИЯ ===
*/

k_SS = round(k_SS * 255.0f) / 255.0f;
float out_gray = saturate(k1 - k_SS + 0.35f);

```

```
    return float4(out_gray, out_gray, out_gray, 1.0f);
}
```

// ГЛАВНЫЙ АППАРАТНЫЙ ШЕЙДЕР ФИЛЬТРАЦИИ

```
float4 PS_Stage3(VS_OUTPUT input) : SV_Target
{
    float2 tex0 = float2(input.Tex.x - 2.0f * d_width, input.Tex.y - 2.0f * d_height);
    int tex_x = round(width * tex0[0] / 4.0f);
    int tex_y = round(height * tex0[1] / 4.0f);

    if (tex_x % 2 == 0)
    {
        if (tex_y % 2 == 0) return func_LT(tex0);
        else return func_LB(tex0);
    }
    else
    {
        if (tex_y % 2 == 0) return func_RT(tex0);
        else return func_RB(tex0);
    }
}
```

// ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ ЭКРАНА С УЧЕТОМ ШТОРКИ ДО/ПОСЛЕ

Texture2D statsTexture : register(t1); // Сюда подадим готовый фильтр из Stage3

```
float4 PS_Final(VS_OUTPUT input) : SV_Target
{
    if (input.Tex.x < splitX)
    {
        // Слева переводим сырой оригинал в честный монохром по вашим весам
        float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
```

```
float src_gray = dot(src_color.rgb, WEIGHT);  
return float4(src_gray, src_gray, src_gray, 1.0f);  
}
```

```
// Тонкая разделительная черная линия
```

```
if (abs(input.Tex.x - splitX) < 0.0015f) return float4(0.0f, 0.0f, 0.0f, 1.0f);
```

```
// Справа выводим чистый результат дефектоскопа
```

```
return statsTexture.Sample(samplerState0, input.Tex);
```

```
}
```

```
// Простой шейдер копирования кадра для ЭТАПА II
```

```
float4 PS_Copy(VS_OUTPUT input) : SV_Target
```

```
{
```

```
    return srcTexture.Sample(samplerState0, input.Tex);
```

```
}
```

```
// ВРЕМЕННАЯ ЗАГЛУШКА ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТА В MAIN.CPP
```

```
float4 PS(VS_OUTPUT input) : SV_Target
```

```
{
```

```
    return srcTexture.Sample(samplerState0, input.Tex);
```

```
}
```

```
// === ОФИЦИАЛЬНЫЕ ЗАГЛУШКИ ДЛЯ ЧИСТОГО СТАРТА ПРОГРАММЫ БЕЗ ОШИБОК ===
```

```
float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target
```

```
{
```

```
    return srcTexture.Sample(samplerState0, input.Tex);
```

```
}
```

```
float4 PS_Stage2(VS_OUTPUT input) : SV_Target
```

```
{
```

```
        return srcTexture.Sample(samplerState0, input.Tex);
    }

float4 PS_PixelShaderStage2(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}
```